后训练学习计划 • 第 14 周（12/7-12/13 • Phase 4 面试火力准备 • 第二周）

本周定位：**刷题推进 + 手撕自证 + 八股第二批 + 把 M2/M3/M4 的坑变成资产**。四件事里没有一件是"学新东西"，全部是把已经踩过的东西变成能在面试桌上掏出来的弹药。 用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	链表 6 题 + 失败案例台账骨架 + RL 基础八股	fail-log.md 模板定稿、链表 6 题	6h
Day 2 周二	二叉树 6 题 + 手撕 1：LoRA 层	test_lora.py 全绿	6.5h
Day 3 周三	BFS/DFS 6 题 + 手撕 2：Adam + 评测方法八股	Adam 对拍 OK	6h
Day 4 周四	回溯 6 题 + 手撕 3：numpy 前向 + 补跑脚本	numpy/torch 块对拍 OK	6.5h
Day 5 周五	混合 6 题 + 八股闭卷自测 + 30 秒讲法录音	累计新题 30、录音 ≥6 条	6h
Day 6 周六	第 2 次模拟面 + 失败案例清单定稿	mock-02 复盘 ≥10 条、fail-log ≥20 条	6.5h
Day 7 周日	补漏 + 闭卷复测 + 总验收 + Week15 预习	验收 ≥6/7 项通过	4.5h

注：**本周卡时上限 4h，预估花费 ¥6-10**（4090 按量 ¥1.5-2.5/时）。这 4h 只为一件事：失败案例清单里某条只有"现象"没有"证据"（例如 M3 的 reward 曲线截图丢了、M2 的 rank 消融没留表），才允许开卡补跑一次小实验。**超了就砍**：补跑改成引用 Week 8/9/12 已存的 wandb 截图或日志片段，条目照写，数字后面标"未量化"，不要为了截图烧钱。求职周不排训练任务。

注：本周最大风险是**刷题变成"看题解会了"**。判据一律用「独立 AC」：提交前没打开题解页、没让 AI 生成解法，只允许查语法。超时规定见 Day 1 任务 1。第二大风险是失败案例清单写成流水账——写不出"30 秒讲法"的条目等于没写。

注：砍单顺序（实在做不完时按这个顺序砍）：Day 4 任务 5 补跑脚本准备 → Day 3 任务 4 评测方法八股（推到 Day 5）→ Day 5 任务 4 台账跟进 → **刷题、手撕、失败案例、模拟面四项不许砍**。这四项正好是面试的四个考察面。

二、开始前：目录、环境、台账与写作规范（约 45 分钟，前一晚做完）

1. 建目录（bash 用正斜杠，反斜杠会被吞）：

```
mkdir -p E:/study/posttrain/week14/{day1,day2,day3,day4,day5,day6,day7}
mkdir -p E:/study/posttrain/llm-posttraining-lab/interview
```

2. 本地环境（手撕三件全在本机 CPU 跑，不需要 GPU）：

```
cd E:/study/posttrain
python -m venv .venv
# PowerShell:  .venv\Scripts\Activate.ps1      Git Bash:  source .venv/Scripts/activate
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
pip install numpy torch
python -c "import numpy, torch; print(numpy.__version__, torch.__version__)"
```

> 注：**Windows + Anaconda 必踩**：`import torch` 抛 `OMP: Error #15: Dependency Vs2015 dll already initialized`（conda 的 MKL 与 torch 自带 OpenMP 抢同一个运行时）。正解是用上面这个独立 venv，别在 conda base 里跑；急着往下做就先 `set KMP_DUPLICATE_LIB_OK=TRUE`（cmd）或 `export KMP_DUPLICATE_LIB_OK=TRUE`（Git Bash）绕过，然后继续。今天 Day 3/Day 4 两个对拍脚本都要 import torch，一定在这个 venv 里跑。

3. 建三份台账（先 `cd E:/study/posttrain/llm-posttraining-lab/interview`；本周所有产出都往这三个文件里落，**不许散落在笔记里**）：
   - `solved.md`：刷题台账，每题一行，格式见 Day 1 任务 1。
   - `fail-log.md`：失败案例清单，条目模板与 6 条样板见下面的「写作规范」。**本周核心资产**。
   - `bagu-2.md`：八股第二批（RL 基础 / reward hacking / 评测方法 / 数据构造），每个主题一张"问题—我的答案—漏洞"三列表。
4. 把 Week 13 的三份东西拉到今天能看见的位置：`week13/week13-gaps.md`（错题与八股第一批漏洞）、简历终版文件、第 1 次模拟面的记录——Day 1 任务 1 和任务 3 直接要用。录音/录屏工具任选一个能用的（手机录音 + 电脑会议软件录屏即可），**别在工具上花时间**。

写作规范：失败案例条目模板 + 6 条样板（读 15 分钟，本周所有条目按这个格式写）

条目模板（五段，粘到 fail-log.md 顶部当规范）：

```
### F-<编号> <一句话标题，≤20 字>
- 阶段与标签：M2 SFT / M2 DPO / M3 GRPO / M4 tutor；标签 [lr] [数据] [mask] [显存] [工程环境] [评测] [reward]
- 现象：报错原文第一行，或曲线的具体形状，或输出样例
- 定位：按什么顺序查（lr → 数值精度 → 数据 → 初始化 → 梯度流），定位到那一步的关键证据是什么（一行日志、一个 shape、一个参数量）
- 修复：改了哪个配置或哪行代码；改后的变化写"量级 + 出处文件路径"，没留记录就写"未量化"
- 30 秒讲法：结论在前的两三句面试口径，朗读 ≤40 秒
- 追问预案：面试官下一句最可能问什么，我怎么答
```

下面 6 条是**格式样板**，用来告诉你"每一段该写多具体、什么算证据"。**其中的数值只是示意，说明这一格该放什么类型的内容——你写自己那份时必须换成你日志/wandb 里的真实记录，示意值一条都不许抄进简历或 fail-log。**

F-01 train loss 三条曲线全平，可训参数量是 0（M2 • SFT） - 现象：LoRA 跑了 300 step，train loss 几乎不动，grad norm 恒为 0，看起来"训练在走但什么都不学" - 定位：按 lr→数值精度→数据→初始化 查；第 0 步先加一行打印 `sum(p.numel() for p in model.parameters() if p.requires_grad)`，结果是 0，再对比模型层名，发现 `target_modules` 里的模块名在这个模型上不存在（不同模型族的投影层命名不一样） - 修复：改成该模型真实的投影名，可训参数从 0 变成百万量级（`出处：m2-sft/README.md` 的参数打印），loss 才开始下降 - 30 秒讲法：我第一次 SFT 训练 loss 完全不动，先在优化器上加了一行可训参数量打印，发现是 0，原因是 LoRA 挂载的模块名写错、一个参数都没注册上。从那以后我的训练脚本第一步永远是打印可训参数表，这是我排查 loss 不降

的第一道闸门。 - 追问预案：会问"怎么快速确认 LoRA 挂上了"——答：`model.print_trainable_parameters()` 或直接遍历 named_parameters 按前缀分组统计；再说 adapter 权重里 B 全零、A 随机，加载后先看 B 的范数是否非零。

F-02 SFT 后模型只会复述问题（M2 • SFT • mask） - 现象：训完的模型对"用一句话解释 XX"这类指令，输出先把问题重抄一遍再空行，eval loss 看着还行 - 定位：拿一个 3 轮对话样本把 label 解码出来打印，发现 prompt 段的 label 不是 -100——数据格式化时走的是"整段拼接再全算 loss"的路径，等于教模型复述指令 - 修复：改用 `tokenizer.apply_chat_template` 生成角色分段，再按 assistant span 置 label，只对 assistant token 算交叉熵 - 30 秒讲法：我最印象深的一次是模型训完只会复读问题。表面看 loss 正常，其实是 label mask 没生效，prompt 也进了 loss。修复之后我把"打印一个样本的 label span"加进了训练前的必查清单。 - 追问预案：会问"为什么只算 answer"——答：instruction 在推理时是已知条件，学它等于浪费容量且教出复述行为；再补一句 packing 的副作用（跨样本注意力泄漏）。

F-03 手拼 chat template 导致推理格式漂移（M2 • SFT • 工程环境） - 现象：训练时输出格式整齐，接上线上推理后模型偶尔多吐一个换行、或把角色标记当正文续写 - 定位：把训练用的字符串和推理侧 `apply_chat_template` 的结果逐字符 diff，发现训练里手拼的模板少了结束标记，两边编码不一致 - 修复：训练与推理统一走 `tokenizer.apply_chat_template`，禁止手拼字符串；重训后 A/B 那 30 个场景的格式非法率显著下降（未量化，看 A/B 表） - 30 秒讲法：我的模板 bug 不是训练崩，是训练和推理两侧各自拼模板、拼得不一样。这类问题曲线看不出来，只有把两边字符串做 diff 才现形，所以我现在的纪律是模板只有一处来源。 - 追问预案：会问"ChatML 风格标记具体是什么"——答 Qwen 系列的角色起始/结束特殊 token，并说明用错 tokenizer 的 template 会让 SFT 白训。

F-04 DPO 里 chosen 和 rejected 的对数概率一起掉（M2 • DPO • reward） - 现象：训练后期 margin（chosen - rejected）在涨，但两条 logprob 曲线都在往下走，chosen 的下降速度比 rejected 慢 - 定位：查 TRL 的指标口径，确认 margin 涨是靠"rejected 掉得更快"实现的；这不是学到偏好，而是同时打压两者 - 修复：把 lr 再降一档、缩短 epoch、混入 SFT 损失（或换 SimPO/KTO 的思路做对照），并把"两条 logprob 必须一起看"写进监控清单 - 30 秒讲法：DPO 最容易骗人的指标是 reward margin。我见过 margin 上升但 chosen 概率也在跌的情况，这时候模型其实是在变得更不擅长说人话，不是在变好。所以我看 DPO 曲线一定看三条：margin、chosen logprob、rejected logprob。 - 追问预案：会问 β 的作用——答：β 控制偏离 ref 的强度，太大学不动、太小易过拟合偏好并让语言退化，我用的量级是 0.1。

F-05 GRPO 组内优势全零，reward 曲线一条直线（M3 • GRPO • reward） - 现象：reward 均值在前若干 step 不升，KL 却慢慢飘高，看起来"在优化但没方向" - 定位：把组内 reward 打印出来，发现同一 prompt 的 G 个采样结果全对或全错，`std` 为 0，`(r_i - mean)/std` 归一化后整组优势为 0（分母处理不同实现会给 0 或小 eps，行为不同，要确认你用的框架） - 修复：提高采样温度或换难度更合适的题（准确落在 0.3-0.7 区间的题最有信息量）、增大 G、丢弃零方差组 - 30 秒讲法：我 GRPO 第一次不涨的原因很简单——题目太简单，全组都做对，组内基线等于零信号。之后我把"题目难度分布"当成 RLVR 的第一超参来调。 - 追问预案：会问"怎么知道该扔掉哪些题"——答：先采一轮统计每题通过率，保留中间段、去掉两端；再说这也解释了为什么规则奖励要设计成部分可给分。

F-06 输出长度暴涨而 reward 平（M3 • GRPO • reward hacking） - 现象：中后期平均输出长度快速上冲、reward 不再涨，抽样看内容是重复句式与堆砌步骤 - 定位：把 reward 拆成子项（正确性 / 格式 / 长度）分别看，发现格式项被"长答案里重复出现合法格式"吃掉；长度分布曲线与 reward 曲线放同一张图就是证据 - 修复：加长度惩罚项或收紧格式校验（只认最后一个答案），必要时重训 RM/验证器；KL 系数调大一点把策略拉回参考附近 - 30 秒讲法：我亲眼见过一次典型 reward hacking：长度暴涨、reward 走平。判断依据是长度与

reward 的解耦，而不是单看 reward 涨没涨。修法要么改奖励函数堵住捷径，要么加 KL 约束。 - 追问预案：会问 KL 系数怎么定——答：没有万能值，我的做法是盯 KL 曲线，健康时受控小幅波动，一旦单调上涨且 reward 不涨就加大约束；这也是为什么我保留了那条不收敛曲线。

> 注：这 6 条样板的共同点是**每条都写到了"我怎么知道的"**，而不只是"我改了什么"。面试里区分度的来源是排查顺序与证据，不是修复动作本身。你写自己那 20 条时，如果某条的"定位"段只有一句"改了配置就好了"，那条等于没写。

Day 1 • 周一：链表 6 题 + 失败案例台账骨架 + RL 基础

今日总览（6h）：晚上 11 点前要拿到两样东西：**solved.md 里 6 行链表题的独立 AC 记录**，和 **fail-log.md 的条目模板 + 前 3 条真实案例**。模板今天不定稿，后面五天的坑就没地方收，Day 6 的定稿任务会变成重新造轮子。

任务 1：链表 6 题，独立 AC（约 150 分钟）

1. 题单（链表部分**进阶组**——Week 13 已经把 206/141/142/160/19/234/146 那批基础题做掉了，今天不重做，全部上难度；括号内是限时）：**92 反转链表 II(15) / 82 删除排序链表中的重复元素 II(20) / 143 重排链表(20) / 25 K 个一组翻转链表(30) / 148 排序链表(25) / 23 合并 K 个升序链表(25)**，限时合计 135 分钟；回炉题（第 3 步）另有 20 分钟——**题单计时之和已经贴着 150 分钟的上限**，超时优先砍回炉题，别砍题单。
2. 规则（**硬性**）：
   - 每题计时开始就写，**25 分钟（Medium 35 分钟）内没 AC 就停笔**，看题解理解后自己重敲一遍，台账里标"抄会"，抄会的不计入 AC 数。
   - 允许查语法（`None` 比较、`ListNode` 定义怎么访问），不允许看解法思路，不允许让 AI 直接给代码。
   - 每题 AC 后立刻做一件别人不做的事：**对着题用 30 秒说出解法与复杂度**（说给自己听，录音也行）。说不清的题，两天后一定会忘。
3. 上周 Week 13 遗留的数组/双指针错题，今天先回炉 1 题（放在链表 6 题之前，20 分钟），从 `week13-gaps.md` 里挑标了"重做"的第一条。**这是本周的 gaps 交接点：上周错题 + 八股第一批（Transformer/训练技巧）的漏洞，本周 Day 3/Day 5 分两次清。**

完成判据：solved.md 新增 6 行（题号 / 第一次用时 / 是否独立 AC / 一句话解法），其中**独立 AC ≥5 题**；第 3 步回炉题重做通过。

> 注：这组是链表里真正区分"写过代码"和"刷过 Hot 100"的部分。`92` 与 `25` 考同一件事：**断点前后的指针怎么接回去**（`92` 要记 pre 与 tail，`25` 要记上一组的尾）；`82` 考虚拟头节点 + "跳过整段相同值"的循环边界；`148` 是链表归并（快慢指针找中点 → 递归拆 → 合并），`23` 要么堆要么分治，两种都写一遍。`143` 重排链表 **是这三招的缝合题**（找中点 + 反转后半 + 交叉合并），面试出镜率比前几道还高。写崩任何一道，第二天闭卷重敲。

任务 2：fail-log.md 建库，先写 3 条完整条目（约 80 分钟）

1. 打开 `E:/study/posttrain/NOTES.md`、`gaps.md`、`llm-posttraining-lab/` 下各 repo 的 README 与 issue 式记录，再翻 Week 8/9/10/11/12 的 wandb 截图目录。**今天不动员"回忆"，只做机械收集**：凡是见过的报错原文、不收敛曲线、奇怪输出，一律先粘进 fail-log.md 的"原始素材池"区，宁滥勿缺，目标 ≥40 行原始记录。
2. 按「开始前」那节的**条目模板**把模板区粘到文件顶部；对照 6 条样板，把今天收集到的素材**先写成 3 条完整条目**（五段齐全）。这 3 条是你自己的密度基准，后面五天不许低于它。
3. 每条打一个标签 `[lr]` `[数据]` `[mask]` `[显存]` `[工程环境]` `[评测]` `[reward]`，Day 6 定稿时按标签统计分布——**分布本身是面试话术**："我踩的 20 个坑里 6 个在数据管线、4 个在 mask，说明我的短板是..."。

完成判据：fail-log.md 有模板区 + 原始素材池 ≥40 行 + 完整条目 ≥3 条（五段齐全）。

> 注：**别把这一项当成"整理笔记"**。这是本周最值钱的东西。后训练岗位的面试流程高度一致：八股 15 分钟 + 项目 20 分钟 + 手写 15 分钟，项目那 20 分钟里，讲"我调了 LoRA 效果不错"的人一批，讲"我 loss 不降，先打印可训参数量发现是 0，因为 target_modules 名字拼错，这条排查顺序我现在背得出"的人极少。企业要的就是后者——训练岗日常就是 debug，不是发明算法。

任务 3：八股第二批启动 • RL 基础（约 120 分钟）

1. 范围（RL 基础，不含后训练专属，那些是第一批已经背过的 DPO/GRPO）：状态/动作/策略/价值/回报的定义与记号；V(s) 与 Q(s,a) 的区别；贝尔曼方程写成一行；on-policy vs off-policy（为什么 PPO 里的概率比 `π_θ/π_old` 就是"半 off-policy"）；优势 A=Q-V；GAE 的直觉（λ 在偏差与方差之间插值，写出递推式）；policy gradient 定理一句话（$\nabla J = E[\nabla \log\pi \cdot A]$）；REINFORCE 为什么方差大；clip 为什么让 PPO 比 vanilla policy gradient 稳。
2. 动作：**不是读，是写**。每个概念在 `bagu-2.md` 里写"我的一句话答案"，然后合上文件闭卷重答一遍，答错的标红。查 `guide-text.txt` 与 Week 9 的笔记核对。
3. 把 GRPO 的组内优势公式手推一遍到纸上：同一 prompt 采 G 个输出，`A_i = (r_i - mean(r))/std(r)`，说清它替掉了 PPO 的哪个网络、代价是什么（要更多采样）。拍照进 repo。
4. 对照 Week 13 八股第一批（Transformer / 训练技巧）的漏洞清单，**今天顺手补 2 条**（第一批遗留的洞在 Day 5 闭卷自测里会被抓出来，现在补比那时补便宜）。

完成判据：bagu-2.md 的 RL 基础表 12 问全部写过答案，其中闭卷重答 ≥9 问正确；纸上 GRPO 优势推导演示照片 1 张入 repo。

> 注：`GAE 的 λ` 是实习面的分界线题——大部分候选人只会背"降低方差"。你要能说出"λ=0 退化成单步 TD（偏差大方差小），λ=1 退化成蒙特卡洛回报（方差大偏差小），实际取 0.95 是折中"，并能写出递推 `δ_t = r_t + γV(s_{t+1}) - V(s_t)`、`A_t = δ_t + γλ A_{t+1}`。语言模型 RLHF 里通常 γ=1、每句一个 reward，把这个特例说清更加分。

任务 4：收工笔记（约 10 分钟）

NOTES.md 写三行：今天最痛的一个 AC 卡点、fail-log 里最想讲的那条是哪条、明天预计开始时间。**完成判据**：NOTES.md 末尾新增 3 行，三行都具体到题号或条目编号（写"感觉还行"不算）。

今日验收：链表 6 行台账（独立 AC ≥5）+ fail-log 模板定稿与完整条目 ≥3 + RL 基础 12 问过一遍。到点没齐就把缺的写进 gaps.md 再睡，熬夜会把 Day 2 的手撕质量拖垮。

Day 2 • 周二：二叉树 6 题 + 手撕 LoRA 层

今日总览（6.5h）：今天的重头是**手撕第 1 件：LoRA 层从零实现 + 单测自证**。面试官问"你会写 LoRA 吗"，能当场写出 `ΔW = BA` 并把初值、缩放、merge 三件事说清的人，比背十遍"低秩假设"的人值钱。

任务 1：二叉树 6 题（约 150 分钟）

1. 题单：**104 二叉树的最大深度(15) / 226 翻转二叉树(20) / 101 对称二叉树(25) / 102 层序遍历(30) / 98 验证二叉搜索树(35) / 236 最近公共祖先(25)**。
2. 规则同昨天：超时看题解就标"抄会"，不计入 AC 数。`98 验证 BST` 必须用**上下界递归**写法（`min/max` 传参），只比父子节点是错的，面试官会当场戳这个点。
3. AC 后用 30 秒说清"这题是 DFS 前序还是 BFS 层序，为什么"。

完成判据：6 行台账，**独立 AC ≥5**；`98` 与 `236` 两题必须是独立 AC（它们是树部分最高频的两道）。

> 注：`236 最近公共祖先` 的递归写法只有 5 行，但要能讲清"为什么左右子树各命中一个就是当前节点"。这题也是追问接口：面试官接着问"如果节点有父指针呢""如果树是二叉搜索树呢"（后者可以用值的大小关系剪枝，复杂度从 O(n) 降到期望 O(h)）。

任务 2：手撕 1 • LoRA 层 + 单测（约 120 分钟）

1. 新建 `week14/day2/lora_layer.py`，接口是 `LoRALinear(base: nn.Linear, r=16, alpha=32, dropout=0.0)` + `forward` + `@torch.no_grad() merge()`（merge 把 ΔW 并进 `base.weight`，之后不再走旁路）。**先自己写 15 分钟**，卡住才看下面的参考实现（重点看初值与形状）：

```
import torch, torch.nn as nn

class LoRALinear(nn.Module):
    """base 冻结 + 低秩旁路。ΔW = B @ A，形状 (out_f, in_f) 与 base.weight 同形"""
    def __init__(self, base, r=16, alpha=32, dropout=0.0):
        super().__init__()
        self.base, self.r, self.scaling, self.merged = base, r, alpha / r, False
        for p in self.base.parameters():
            p.requires_grad = False                              # 基座冻结
        in_f, out_f = base.in_features, base.out_features
        self.A = nn.Parameter(torch.randn(r, in_f) * 0.05)       # A: (r, d_in) 随机初始化
        self.B = nn.Parameter(torch.zeros(out_f, r))             # B: (d_out, r) 初值必须为 0
        self.drop = nn.Dropout(dropout)

    def forward(self, x):
        y = self.base(x)
        if self.merged:
            return y
        return y + self.drop(x) @ self.A.t() @ self.B.t() * self.scaling

    @torch.no_grad()
    def merge(self):
        self.base.weight.data += self.scaling * (self.B @ self.A)
        self.merged = True
```

2. 新建 `test_lora.py`，**自己先写这 4 个断言**，写不出再对照：① 初始 ΔW=0，未训练时输出与 base 一致；② 可训参数量 == `r*(in+out)` 且只有 A、B 可训；③ merge 前后输出 allclose（float64，atol=1e-5）；④ backward 后梯度只出现在 A、B 上。

```
import torch, torch.nn as nn
from lora_layer import LoRALinear

torch.manual_seed(42)                    # 种子写死：对拍要同一组权重，否则失败无法复现
base = nn.Linear(64, 32).double()
layer = LoRALinear(base, r=8, alpha=16).double()
x = torch.randn(5, 64, dtype=torch.float64)

assert torch.allclose(layer(x), base(x), atol=1e-9); print("zero-init OK")
tr = {n for n, p in layer.named_parameters() if p.requires_grad}
assert tr == {"A", "B"} and sum(p.numel() for p in layer.parameters() if p.requires_grad) == 8 * 96
print("trainable-count OK")
with torch.no_grad():
    layer.B.normal_(0, 0.1)
ref = layer(x).clone(); layer.merge()
assert torch.allclose(base(x), ref, atol=1e-5); print("merge-equiv OK")
b2 = nn.Linear(64, 32).double(); l2 = LoRALinear(b2, r=8, alpha=16).double()
l2(x).pow(2).mean().backward()
assert l2.A.grad is not None and l2.B.grad is not None
assert all(p.grad is None for p in l2.base.parameters()); print("grad-only-ab OK")
print("LORA TESTS OK")
```

3. 跑：`cd E:/study/posttrain/week14/day2 && python test_lora.py`（在 venv 里）。

4. 顺手给 fail-log 补 2 条（写 LoRA 时想起 M2 的 rank 配置，就补一条 `[lr]` 或 `[mask]` 类）。

完成判据：终端打印 4 个测试名 + `LORA TESTS OK`；限时达标——从空文件到全绿 **≤45 分钟**（面试手撕的实况时限），超时就把卡住的那一步写进 gaps，明天再来一轮。

> 注：三处必被追问，答案要提前想好：① **B 为什么初值 0**——保证训练起点 ΔW=0，不破坏基座能力，A 若为 0 也一样，但论文让 A 随机初始化以提供梯度（B=0 时 A 拿不到梯度，所以必须随机 A + 零 B）。② **alpha 和 scaling 干什么**——scaling=alpha/r，让不同 rank 下旁路更新的幅度可比，换 rank 不必重调 lr。③ **merge 之后怎么回滚**——**merge 前必须留未 merge 的 adapter 与基座权重**，回滚就是重新加载基座；面试里"LoRA 回滚成本低"的标准答法是：adapter 只有几 MB，切回来只是换个目录，全量微调的回滚是重训。

任务 3：八股 • reward hacking（约 60 分钟）

1. 在 `bagu-2.md` 写"reward hacking"专节，必须覆盖：定义（模型找到骗 RM 的捷径而非真变好）；**三种发现手段**（输出长度分布突变、KL 爆走、人工抽检质量下降）；**四种缓解**（KL 惩罚/β、RM 加长度惩罚项、定期用新策略数据重训 RM、RM 集成）；RLVR 场景为什么也存在 hacking（规则奖励可被格式钻空：只输出答案不输出推理也能拿分、单位/取整投机）。
2. 把 M3 里你**真实观察到**的那次长度或格式异常写成 fail-log 一条（Day 3 会集中整理 M3 段）。
3. 准备一句自己的话版本，别背教材句。

完成判据：bagu-2.md 该节 ≥6 条自问自答且闭卷重答全对；fail-log 新增 reward hacking 相关条目 ≥1 条（五段齐全）。

> 注：这题的加分答案是"我自己怎么发现的"：GRPO 里 reward 上升同时**平均输出长度上升、正确率不变**，这个组合就是 hacking 信号——把它写成一条曲线读法："reward 与长度同向上、reward 与准确率脱钩"。面试官会立刻知道你跑过真实验。

任务 4：失败案例 • M2 段起 3 条 + 收工（约 60 分钟）

1. 翻 `m2-sft/` 的 README、Week 6/8 的 gaps 条目与当时的截图目录，按模板写 3 条（标签集中在 `[mask]` `[数据]` `[lr]`），Day 3 的 M2 段任务在此基础上补到 ≥5 条。
2. 每条写完朗读计时一遍，超 40 秒就删修饰语。
3. 从 `week13-gaps.md` 挑 1 道数组/双指针错题闭卷重敲（15 分钟）；NOTES.md 写今天 LoRA 卡在哪一步、明天 Adam 预估耗时（5 分钟）。

完成判据：M2 段新增 3 条、五段齐全，朗读秒数标在条目末尾且 ≤40 秒；回炉错题 1 道打 `[x]`。

今日验收：树 6 行台账（独立 AC ≥5）+ `LORA TESTS OK` 且 ≤45 分钟 + reward hacking 节写完 + fail-log 累计 ≥10 条。

Day 3 • 周三：BFS/DFS 6 题 + 手撕 Adam + 评测方法

今日总览（6h）：图搜索一组 + 手撕第 2 件（Adam 手写 + 与 torch 对拍）。Adam 这件的判据不是"写出来"，是**逐迭代参数对齐到 1e-6**——这是唯一能证明你真的懂偏差修正和权重衰减位置的办法。

任务 1：BFS/DFS 6 题（约 140 分钟）

1. 题单：**200 岛屿数量(20) / 994 腐烂的橘子(25) / 133 克隆图(20) / 207 课程表(30) / 547 省份数量(25) / 208 实现 Trie 前缀树(20)**。
2. 规则：同前两日。`207 课程表` **要写两种**——BFS 拓扑排序（入度数组）与 DFS 三色标记判环，第二种 15 分钟内补写，这是高频追问。
3. `994` 的坑在"结束时还有新鲜橘子就返回 -1"，写完自己造一个孤立橘子的用例验证。

完成判据：6 行新题台账，**独立 AC ≥5**；`207` 的 BFS 拓扑排序与 DFS 三色标记两种写法都能不看资料敲出来；`208` 的 `insert/search/startsWith` 三个方法都通过自己造的用例。

> 注：图题在后训练面试里出现频率不高，但 `207 课程表` 的拓扑排序思想会换个马甲问："多阶段训练（通用 SFT → 领域 SFT → 偏好对齐 → 安全对齐）为什么不能并行？"——因为梯度目标互相冲突，有依赖顺序。这个类比是你独有的加分答案，写进 `bagu-2.md` 的"后训练为什么要分阶段"条目旁边。

任务 2：手撕 2 • Adam/AdamW 手写 + 对拍（约 120 分钟）

1. 新建 `week14/day3/my_adamw.py`：

```
import numpy as np

def adamw_step(params, grads, m, v, t, lr=1e-3, b1=0.9, b2=0.999, eps=1e-8, wd=0.01):
    """原地更新 params。t 从 1 开始计数。wd 是解耦权重衰减(AdamW),不是 L2 进梯度"""
    for i in range(len(params)):
        m[i] = b1 * m[i] + (1 - b1) * grads[i]
        v[i] = b2 * v[i] + (1 - b2) * grads[i] ** 2
        m_hat = m[i] / (1 - b1 ** t)              # 偏差修正
        v_hat = v[i] / (1 - b2 ** t)
        params[i] -= lr * (m_hat / (np.sqrt(v_hat) + eps) + wd * params[i])
```

2. 新建 `week14/day3/test_adamw.py`：同一个二次函数 + 同一个初始点，float64，跑 20 步，与 `torch.optim.AdamW`（`betas=(0.9,0.999), eps=1e-8, weight_decay=0.01, lr=1e-3`）逐步对拍。

```
import numpy as np, torch
from my_adamw import adamw_step

np.random.seed(0); torch.manual_seed(0)          # 两边共用同一个初始点，否则无从对齐
target = np.array([1.0, -2.0, 0.5, 0.0])
w_np = np.random.randn(4).astype(np.float64)
w = torch.tensor(w_np.copy(), requires_grad=True)
tgt = torch.tensor(target, dtype=torch.float64)
opt = torch.optim.AdamW([w], lr=1e-3, weight_decay=0.01)
m, v = [np.zeros_like(w_np)], [np.zeros_like(w_np)]
max_diff = 0.0
for t in range(1, 21):                           # 同一个二次目标，梯度 = w - target
    adamw_step([w_np], [w_np - target], m, v, t)
    opt.zero_grad(); (0.5 * (w - tgt).pow(2).sum()).backward(); opt.step()
    max_diff = max(max_diff, np.abs(w_np - w.detach().numpy()).max())
    print(t, max_diff)
assert max_diff < 1e-6, f"AdamW 对拍失败 max_diff={max_diff}"
print("ADAMW CROSSCHECK OK")
```

3. 跑 `python test_adamw.py`。**必须 float64**：float32 下 20 步后误差会到 1e-7~1e-6 边缘，判据失去意义。
4. 在 NOTES.md 写三行：① 不加偏差修正前几步更新会怎样（方向对但步长偏小还是偏大？自己跑一遍 `b1_hat=b2_hat=1` 的对照组）② AdamW 与 Adam+L2 的一行差别 ③ 为什么后训练常配 warmup + cosine。

完成判据：打印 `ADAMW CROSSCHECK OK`（max_diff < 1e-6）；偏差修正对照组跑出可复现结论一行（写进 NOTES）；限时达标——**从空文件到 OK ≤30 分钟**（Day 7 闭卷复测就按这个时限）。

> 注：面试写 Adam 最常犯的两个错：① 把 `wd` 混进梯度（那是 Adam+L2，AdamW 是解耦的 `p -= lr*wd*p`，两者在自适应缩放下不等价）；② `eps` 放在分母根号外还是内——PyTorch 的实现是 `sqrt(v_hat) + eps`，写成 `sqrt(v_hat + eps)` 对拍不上，这不是谁错，是实现约定，能讲清这个细节就是加分。

任务 3：失败案例 • M2 段整理（约 50 分钟）

1. 只处理 M2（SFT + DPO）：翻 `m2-sft/` 的 README、wandb 记录、`gaps.md` 里 Week 6/8 的条目。目标 **5-6 条**完整条目，标签集中在 `[mask]` `[数据]` `[lr]`。
2. 每条写完后，**把 30 秒讲法朗读计时一遍**，超过 40 秒就删修饰语、只留"结论 + 证据 + 动作"。

完成判据：M2 段条目 ≥5 且五段齐全；朗读计时全部 ≤40 秒（记录实际秒数在条目末尾）。

> 注：M2 里必有一条是 mask：SFT 的 label 规则是 prompt 部分置 -100、只在 assistant token 上算交叉熵。当时你如果出现过"模型学会了复述问题"，那就是 mask 没生效的铁证，写进去——这个现象-根因配对是面试杀手锏，因为它只有真训练过的人才讲得出。

任务 4：八股 • 评测方法（约 50 分钟）

1. `bagu-2.md` 评测节覆盖：公开基准（lm-evaluation-harness、OpenCompass）跑什么、防能力退化怎么看；自建评测集怎么定 rubric；LLM-as-judge 的三条纪律（**固定 rubric、固定温度、同一样本重跑 3 次做**

一致性抽查）；偏好对 held-out 准确率（chosen 概率高于 rejected 的比例）是什么、它能说明什么不能说明什么；GSM8K 的 final-answer 抽取规则为什么会吃掉一半错误。

2. 背一段可执行命令（Day 4 补跑任务会真用到）：

```
lm_eval --model hf --model_args pretrained=./merged-tutor --tasks gsm8k --batch_size auto
```

完成判据：评测节 ≥7 条自问自答、闭卷重答 ≥6 对；能默写出上面这条命令的关键参数（`pretrained` / `tasks` / `batch_size`）；NOTES.md 两行收工笔记（Adam 对拍 max_diff 的实际量级、明天 numpy 前向预计用时）。

> 注：judge 的一致性问题一定会被追问："你信大模型打分？"标准答案不是信或不信，而是**流程**：rubric 写死、温度设 0 或固定采样、抽 20 条人工复核给一致率、和人工分做相关性对照。你在 M4 做过 200 题自建集，那条数据（一致率的量级）直接引到你的答案里。

今日验收：BFS/DFS 有效记录 6 行（独立 AC ≥5）+ `ADAMW CROSSCHECK OK` + M2 段 ≥5 条 + 评测节写完。

Day 4 • 周四：回溯 6 题 + 手撕 numpy 前向 + 补跑脚本准备

今日总览（6.5h）：回溯是刷题里最吃"状态机清晰度"的一组，也是手撕第 3 件（numpy 写一个完整 decoder block 前向，并和 torch 对拍）的日子。今天还要把 Day 里可能开卡 2h 的**补跑脚本写完但不跑**——脚本没写好就开卡，钱就是白烧。

任务 1：回溯 6 题（约 140 分钟）

1. 题单：**78 子集(20) / 46 全排列(25) / 22 括号生成(20) / 39 组合总和(25) / 17 电话号码的字母组合(15) / 79 单词搜索(35)**。
2. 写每题前先默念三段式并写在纸角：**选择 → 递归 → 撤销选择**。`79` 的撤销就是把格子改回临时标记（别用额外 visited 数组也能过，但面试要讲清为什么用标记位更省）。
3. 每题记"第一次 AC 用时"和"状态树大小直觉"（`46` 是 n!、`78` 是 2^n），复杂度说得出。

完成判据：6 行台账，**独立 AC ≥5**；`78`、`46`、`22` 三道独立 AC（这三道是回溯的地基）。

> 注：回溯在面试里常以"变体"出现：给一个 prompt 模板集合让模型批量生成，问你怎么枚举。本质是同一棵树。把 `39` 组合总和 的"可重复选 vs 不可重复选"差别讲清（`start` 是传 `i` 还是 `i+1`），这是最常被戳的点。

任务 2：手撕 3 • numpy 写 decoder block 前向（约 110 分钟）

1. 新建 `week14/day4/block_np.py`。要求：RMSNorm → 多头因果自注意力 → 残差 → RMSNorm → SwiGLU 式 FFN → 残差，形状 (T, C)，**float64**，种子写死。

```
import numpy as np

T, C, H, FF = 6, 16, 4, 64
DK = C // H

def silu(z): return z / (1.0 + np.exp(-z))

def rmsnorm(x, w, eps=1e-6):
    return x * w / np.sqrt((x * x).mean(-1, keepdims=True) + eps)

def softmax(x):
    x = x - x.max(-1, keepdims=True)
    e = np.exp(x)
    return e / e.sum(-1, keepdims=True)

def mha(x, p):
    q, k, v = x @ p["Wq"], x @ p["Wk"], x @ p["Wv"]
    mask = np.triu(np.full((T, T), -np.inf), k=1)
    outs = []
    for h in range(H):
        s = lambda t: t[:, h*DK:(h+1)*DK]
        a = softmax(s(q) @ s(k).T / np.sqrt(DK) + mask)
        outs.append(a @ s(v))
    return np.concatenate(outs, axis=-1) @ p["Wo"]

def block(x, p):
    x = x + mha(rmsnorm(x, p["ln1_w"]), p)
    h = rmsnorm(x, p["ln2_w"])
    return x + (silu(h @ p["Wg"]) * (h @ p["Wu"])) @ p["Wd"]

def make_params():
    r = np.random.default_rng(42)            # 写死：对拍要同一组权重
    W = lambda a, b: r.normal(0, 0.02, (a, b))
    return dict(ln1_w=np.ones(C), ln2_w=np.ones(C), Wq=W(C, C), Wk=W(C, C),
                Wv=W(C, C), Wo=W(C, C), Wg=W(C, FF), Wu=W(C, FF), Wd=W(FF, C))
```

2. 新建 `week14/day4/test_block.py`，三个断言：① 输出形状 (T, C)；② **因果性**：把 `x[T-1]` 换成随机值，前 T-1 行输出与换之前逐元素相等（atol=1e-12）；③ 与 torch 对拍：`max_abs_diff < 1e-9`。

```
import numpy as np, torch
from block_np import block, make_params, T, C, H, DK

def torch_block(x, p):                          # 同一套算式用 torch float64 再写一遍
    t = lambda a: torch.tensor(a, dtype=torch.float64)
    x, p = t(x), {k: t(v) for k, v in p.items()}
    rms = lambda v, w: v * w / torch.sqrt((v * v).mean(-1, keepdim=True) + 1e-6)
    silu = lambda z: z / (1 + torch.exp(-z))
    mask = torch.triu(torch.full((T, T), float("-inf"), dtype=torch.float64), 1)
    def mha(v):
        q, k, vv = v @ p["Wq"], v @ p["Wk"], v @ p["Wv"]
        s = lambda z: z[:, h*DK:(h+1)*DK]
        outs = []
        for h in range(H):
            a = torch.softmax(s(q) @ s(k).T / float(DK) ** 0.5 + mask, dim=-1)
            outs.append(a @ s(vv))
        return torch.cat(outs, -1) @ p["Wo"]
    v = x + mha(rms(x, p["ln1_w"]))
    hh = rms(v, p["ln2_w"])
    return (v + (silu(hh @ p["Wg"]) * (hh @ p["Wu"])) @ p["Wd"]).numpy()

p = make_params(); x = np.random.default_rng(1).normal(size=(T, C))
y = block(x, p)
assert y.shape == (T, C); print("shape OK")
x2 = x.copy(); x2[-1] = np.random.default_rng(99).normal(size=C)
assert np.allclose(y[:-1], block(x2, p)[:-1], atol=1e-12); print("causal OK")
d = np.abs(y - torch_block(x, p)).max(); print("max_abs_diff =", d)
assert d < 1e-9; print("BLOCK CROSSCHECK OK")
```

3. 跑 `cd E:/study/posttrain/week14/day4 && python test_block.py`，期望打印 `shape OK` / `causal OK` / `BLOCK CROSSCHECK OK`，并把 max_abs_diff 的实际量级记进 NOTES。

完成判据：三个 OK 全部打印；**从空文件到全绿 ≤40 分钟**；NOTES 里写清"多头是怎么切的（reshape 还是列切片）""为什么 mask 必须在 softmax 之前"。

> 注：这份 numpy/torch 两边故意写成同一套算式，对拍对的是**实现一致性**，不代表某个真实模型的结构（Qwen/LLaMA 的 FFN 维度比例、是否带 bias、有没有 RoPE 各不相同——这里连位置编码都没写）。面试被问到就讲"这是我为对拍写的最小可比版本，位置编码交给 embedding"，比假装它是 LLaMA 稳。

任务 3：失败案例 • M4 段整理（约 50 分钟）

1. 只处理 M4（tutor 项目）：翻 `m4-capstone/` 的 A/B 协议记录、评测脚本改动、vLLM 起服务的日志。目标 4-5 条，标签多为 `[评测]` `[工程环境]` `[mask]`。
2. 至少一条要写"成本/延迟"相关的实测口径：A/B 表里 API 版与小模型版的延迟与成本对比，数值只写你表里真有的，没有就写"未量化"。

完成判据：M4 段条目 ≥4 且四段齐全；其中 ≥1 条包含"这个数字我是从哪个文件抄的"的出处路径。

任务 4：八股 • 数据构造（约 50 分钟）

1. `bagu-2.md` 数据节：self-instruct 与 evol-instruct 的差别（深度演化 vs 广度演化）；合成数据质量控制四板斧（MinHash-LSH 去重、困惑度双向过滤、奖励模型打分、规则过滤）；去污染（与评测集做 13-gram 交集剔除）；data card 必含字段；PII 脱敏（正则 + 人工抽检）。
2. 写一段"我构造 2000-4000 条教学对话数据时的过滤流水线"，按**顺序**写：生成 → 规则过滤 → 去重 → PPL 过滤 → RM 打分 → 去污染 → 抽样人审。**顺序就是答案**，很多人会说反（先去重后过滤等于白算）。
3. 准备追问："合成数据会不会自我强化偏差？"——答法：生成模型上限决定 + 分布坍缩（模板重复），所以要有 PPL 下限过滤与人工抽检比例。

完成判据：数据节 ≥8 条自问自答且流水线顺序能闭卷写出；`fail-log` 里补 1 条数据类案例（M2/M3/M4 任一段）。

> 注：去污染是"岗位真实日常"的证据，也是最好拿的一题：一定要说出**评测集任何形式都不能进训练集**，以及 n-gram 交集这个可执行判据。有人问到就补一句"我是先算交集再决定阈值，不是拍脑袋"。

任务 5：补跑脚本准备（约 40 分钟，今天不跑）

1. 挑出失败案例里 **最需要一张图来撑场**的一条（例如 M3 的 reward 曲线截图丢了、M2 的 rank 消融只留了结论），在 `week14/day5/rerun.sh` 里把要跑的命令、数据路径、输出目录、预计用时写全。**写脚本，不开卡**。
2. 记一笔账：这条补跑预计消耗多少卡时（例如 `1.5B LoRA + 500 条数据，预计 1.2-1.8 卡时`），落在本周 4h 上限内才允许明天开。
3. NOTES.md 收工两行：block 对拍 max_diff 的实际值、明天最可能超时的任务。

完成判据：`rerun.sh` 存在且**每一行命令都可直接执行**（含 `cd`、模型路径、`seed`），文件头写明预计卡时与要产出的那张图的用途；NOTES 收工两行已写。

> 注：**先 1.5B 跑通再上 7B**，这条纪律本周仍然生效。4090 24G 上 1.5B LoRA(bf16) 舒适，7B LoRA 得 4bit QLoRA，7B 全量单卡不可行——别在求职周临时改配置烧钱。AutoDL 离开终端前确认控制台显示"已关机"，**任何时候不点销毁**（销毁连数据盘一起清空）。

今日验收：回溯 6 行（独立 AC ≥5）+ `BLOCK CROSSCHECK OK` + M4 段 ≥4 条 + 数据节写完 + rerun.sh 可执行。

Day 5 • 周五：混合 6 题 + 八股闭卷自测 + 30 秒讲法录音

今日总览（6h）：刷题收尾到 30 题（累计 ≥55）。今天的关键动作是**开口**：把失败案例条目一条条讲出来并录音。写下来和讲出来是两个能力，面试考后者。

任务 1：混合 6 题（链表/树压轴）（约 150 分钟）

1. 题单（都是各家面经里的常客）：**42 接雨水(35) / 96 不同的二叉搜索树(30) / 105 从前序与中序遍历构造二叉树(25) / 230 二叉搜索树中第 K 小的元素(25) / 124 二叉树中的最大路径和(20) / 114 二叉树展开为链**

表(15)；做不完的那道推到 Day 6 的回炉位。

2. `42 接雨水` 三种解法（双指针 / 单调栈 / 前缀最大值数组）先写双指针那一种，**面试里能讲"为什么左右指针谁小谁先动"才是这题的分。** `96` 是卡特兰数式 DP，`g(i)` 以根为界左乘右，说得出为什么。`230` 用中序遍历（迭代 + 栈，别只写递归）。
3. 今天做完，本周新题数达到 **30**，`solved.md` 顶部累计数应为 **≥55**（Week 13 的 25 + 本周 30）。

完成判据：6 行新题台账 + 任意一道前四天"抄会"题重做独立 AC，**当天独立 AC ≥4**（题单 6 道里做不完的那道算进 Day 6 回炉位，不计入本周新题）；`solved.md` 顶部写明"本周新题 30、累计独立 AC ≥55"并给出你自己的统计式（例如 `grep -c "独立" solved.md`）。

> 注：`105 前序+中序构造` 是"你会不会讲递归"的照妖镜：要能说清前序的第一个是根、在中序里定位根就能切分左右子树区间，以及为什么需要哈希表存中序下标把查找降到 O(1)（否则每层 O(n) 退化）。这题写不出递归边界的一律重做。

任务 2：八股第二批闭卷自测（约 70 分钟）

1. 合上所有资料，在 `bagu-2.md` 末尾的"闭卷 14 问"区作答（四个主题各 3-4 问，问题自己从台账里挑，必须包含这 4 问）：
   - GAE 里 λ 的作用与两个极端；
   - reward hacking 怎么发现、怎么缓解；
   - LLM-as-judge 的三条纪律；
   - 合成数据过滤流水线的顺序。
2. 计时 45 分钟写完，剩 25 分钟对答案并给每题打分（对/半对/错）。**错的题当天写进 `gaps.md`，映射到 Week 15 的具体日子。**

完成判据：14 问全部作答，**≥10 问完全正确**；错的题全部有"补在哪天 + 用什么动作补"。

任务 3：失败案例 30 秒讲法录音（约 80 分钟）

1. 打开 `fail-log.md`，挑出你认为最能打的 **10 条**，逐条朗读计时（**每条 ≤40 秒**），录音存 `week14/day5/talk/`（文件名 `F-07.m4a` 这样对齐条目编号）。
2. 回听全部录音，把每条标一个判定：`可用` / `太长` / `逻辑跳` / `术语堆太多`。**"太长"和"逻辑跳"的当天重写**——删修饰语、把结论提到第一句。
3. 找一条最抽象的（比如 reward hacking 那条）加一个具体证据："第 200 step 之后平均长度涨了将近一倍，正确率没变"，量级说"预期/大约"，**没有记录就写"未量化"，不许编精确值。**

完成判据：录音文件 ≥6 个且每条时长 ≤40 秒；fail-log.md 每条对应位置有判定标记；至少 6 条标为 `可用`。

> 注：为什么卡 40 秒：面试官的注意力窗口就是 30-60 秒，而且你讲完他会打断。练成"三句结构"（我看到什么 / 我怎么定位 / 我改了什么、结果如何）比练成"讲全"更重要。**讲全是书面能力，讲清是面试能力**，这一天练后者。

任务 4：投递台账跟进（约 50 分钟）

1. 打开 Week 13 建的三档目标清单（大厂实验室 / 模型公司 / 业务侧 AI 团队），把已经投出去的家逐条更新状态：`已投` / `笔试` / `约面` / `无回复`。**今天不新增投递**，Week 15 集中投 ≥20 家，今天只把已投的和内推触达记清。
2. 内推渠道记录：每家的渠道写清来源（官网 / 牛客内推码 / BOSS 直聘 / 实习僧 / 即刻或小红书员工），**内推码本身要留档**（很多厂的日常实习内推常年开放）。
3. 给每家补一栏"我这份简历投它时该改哪一句"——这是 Day 6 模拟面之外的个性化功课，Week 15 执行投递时直接照这一栏改。

完成判据：台账文件里已投/已触达 ≥8 家且状态为本周更新；每家有一栏"针对性改动一句话"。

> 注：投递节奏按你手里资料的建议走：**月之暗面 / 智谱 / 美团优先**（后训练方向明确、对无论文友好），字节 / 腾讯次之（用竞赛或开源补背书），百度 / 阿里作为冲刺项。业务侧 AI 团队（教育、出海、座舱）门槛更友好，是第一个实习最现实的跳板，别只盯实验室。

任务 5：收工笔记（约 10 分钟）

NOTES.md：录音里自己最不满意的一条 + 明天模拟面要用的 3 条案例编号。**完成判据**：两行都写了（录音条目标编号、案例编号写出 F-xx），且明天模拟面的抽题脚本已能跑出 8 问。

今日验收：本周新题累计 30、累计独立 AC ≥55 + 闭卷 14 问 ≥10 对 + 录音 ≥6 条且 6 条"可用" + 台账 8 家有状态。

Day 6 • 周六：第 2 次模拟面 + 失败案例清单定稿

今日总览（6.5h）：本周两次大考的日子。**模拟面必须先面再改**——清单没定稿也要面，那才是真实战场状态。下午把 fail-log 定稿到 ≥20 条。

任务 1：未 AC 与"抄会"题回炉（约 90 分钟）

1. 从 `solved.md` 里筛出本周所有标"抄会"或"未 AC"的题，闭卷重敲。每题限时 25 分钟。
2. 重做通过的改标"独立 AC（隔日重做）"，仍不过的留到 Week 15 的复习位。

完成判据：回炉 ≥4 题，其中**重做通过 ≥3 题**；solved.md 状态同步更新。

任务 2：第 2 次模拟面（约 100 分钟，含前置 10 分钟）

1. 形式（按优先级取一个能落地的）：牛客约互面 > 同学/同行互面 > 对镜自述录屏。**必须真人提问**优先——自己给自己出题的模拟面对"被打断"没有抵抗力。找不到人就用对镜录屏 + 从 `bagu-2.md` 里随机抽题（抽题脚本你自己写，`random.seed` 写死，保证可复盘）。

2. 结构（计时执行，不许拖，总时长约 88 分钟）：自我介绍 60 秒 + 项目快讲（8 分钟）→ 八股抽 8 问：RL 基础 + 评测 + 数据构造（20 分钟）→ 项目深挖：抽 fail-log 里 3 条讲（15 分钟）→ 手撕：现场写 LoRA 层或 Adam，二选一（25 分钟）→ LeetCode 一道 Medium，从本周没重做过的错题里挑（15 分钟）→ 反问（5 分钟）。
3. 全程录屏/录音。手撕那 25 分钟**不查任何资料、不许运行调试超过 2 次**（面试环境通常没有 IDE 加持）。

完成判据：模拟面**录屏或录音文件存在**（文件名 `mock-02-<日期>`，时长 ≥50 分钟，落到 `week14/day6/`）；6 段都执行到（缺任一段记为"未完整"，Week 15 的三面补上）。

任务 3：模拟面复盘（约 70 分钟）

1. 回听/回看，逐条记录：每题**我当时说了什么**（尽量原话）、**正确/更优的说法**、**下次怎么说**（≤3 句）。这份记录用 Week 16 要正式的"面试后复盘模板"的雏形，今天就按这三列写。
2. 额外记两项"过程指标"：① 被打断次数 ② 出现"嗯…就是…"式空转的时长。这两个指标是成长速度的直接读数。
3. 复盘条目落 `interview/mock-02.md`。

完成判据：`mock-02.md` 有 **≥10 条**三列复盘；写明被打断次数与空转时长；至少 2 条要落到"改简历/改讲述稿"的具体动作。

> 注：这一轮最容易暴露的不是知识，而是**项目讲述的顺序**：一上来讲"我用了 LoRA 和 DPO"就死。正确顺序是动机 → 方案 → 数据 → 指标 → 失败尝试 → 改进，把失败放在第五位而不是第一位，是因为要先让对方相信你把事做成了，再展示你会 debug。Week 15 的 15 分钟讲述版就是练这个顺序。

任务 4：fail-log.md 定稿（约 100 分钟）

1. 补齐到 **≥20 条**（M2 段 ≥5、M3 段 ≥5、M4 段 ≥4、其余为工程环境/数据/评测类），每条五段齐全（含 30 秒讲法与追问预案）。
2. 文件头加两个小东西：① 标签分布计数（`[数据] n 条 / [mask] n 条 / ...`）② "按重要性排序的前 6 条"索引（面试时间有限，只讲得完 6 条）。
3. 交叉核对：**简历里出现的每一个数字都必须能在 fail-log 或 repo 里找到出处**。找不到出处的数字，删掉或改成量级表述。
4. 把清单 push 进 `llm-posttraining-lab`（`interview/` 目录），本地留一份。

完成判据：fail-log.md 条目 ≥20 且五段齐全率 100%；标签计数与"前 6 条索引"存在；抽查 3 条的数值出处路径可打开；已 push。

> 注：清单里一定要有一条**至今没完全解决**的问题（例如 GRPO 不稳、judge 与人工一致率一般）。面试官最爱问"还有什么你没搞定"，答"没有"等于答"我没真做过"。诚实写"我的判断是…但没验证"，比硬编一个漂亮结论稳。

任务 5：简历 60 秒口播 + 周账（约 30 分钟）

拿着简历终版，计时 60 秒讲完"我是谁 + 我的三个项目各一句 + 我要什么岗"，录一遍回听。**超过 75 秒就删内容，不要加快语速**。然后在 `week14/review.md` 记两行：本周实际用时（对照 42h）、实际卡时与花费（对照 4h/¥6-10 上限）。

完成判据：录音存在，时长 ≤75 秒，三项目各一句不缺；review.md 两行账写完。

今日验收：回炉 4 题 + 模拟面录屏（≥50 分钟，6 段执行）+ mock-02 复盘 ≥10 条 + fail-log ≥20 条定稿 + 简历口播 ≤75 秒。

Day 7 • 周日：补漏 + 闭卷复测 + 总验收 + Week15 预习

今日总览（4.5h）：不排新内容。上午清洞与复测，中午做总验收，下午 30 分钟预习 Week 15（项目深挖 + 集中投递），然后休息——下周是投递强度最大的一周，把状态留给它。

任务 1：补漏（约 60 分钟）

打开 `gaps.md` 和 `week13-gaps.md`，按优先级清洞。**规则：最多补 2 个洞，贪多必失**。优先级：八股第二批答错的题 > 手撕超时的步骤 > 未 AC 的题。**完成判据**：2 个洞标"已清 + 清法"，其余每条都写了目标周与动作。

任务 2：闭卷复测 3 题（约 50 分钟）

补完洞当天就重做，**只看题面、合上所有资料**：

1. 手写 Adam 更新式与 AdamW 的区别（纸面 5 分钟）+ 手敲 `my_adamw.py` 关键 6 行，`ADAMW CROSSCHECK OK` 复现（≤20 分钟，含跑）。
2. 纸上写 GRPO 组内优势公式与"它替掉了 PPO 的哪个网络"，并写出健康/病态曲线各两种（reward 上升 + KL 受控 = 健康；长度暴涨而 reward 平 = hacking）。
3. 口述一条今天新写的失败案例（不看清单），录音 ≤40 秒。

完成判据：三题全部完成；第 1 题在 20 分钟内跑通；第 2 题四个曲线判据全对；第 3 题录音存在且时长 ≤40 秒。

> 注：复测必须**当天做在补漏之后**，隔天就失效——补漏时你看到的是解释，复测时你面对的是空白编辑器。隔一夜剩下的大概率只有"我那天看懂了"的错觉。20 分钟这条线也不是随便定的：面试的手撕环节给的就是 15-25 分钟。

任务 3：本周总验收（约 30 分钟）

逐项勾选，结果写进 `week14/review.md`：

#	验收项	判据
1	刷题	本周新题 ≥30，其中独立 AC ≥25；`solved.md` 累计独立 AC ≥55

#	验收项	判据
2	手撕 ML	LoRA 层 / Adam / numpy 前向三件单测全绿（`LORA TESTS OK`、`ADAMW CROSSCHECK OK`、`BLOCK CROSSCHECK OK`），≥2 件在限时内完成
3	失败案例	`fail-log.md` ≥20 条、五段齐全、含 30 秒讲法与追问预案；录音 ≥6 条 ≤40 秒；已 push
4	八股第二批	RL 基础 / reward hacking / 评测方法 / 数据构造四节写完，闭卷 14 问 ≥10 对
5	模拟面	第 2 次模拟面录屏存在（≥50 分钟、6 段执行）+ `mock-02.md` 复盘 ≥10 条
6	求职资产	`solved.md` / `fail-log.md` / `bagu-2.md` 三台账本周有更新；简历数字与清单出处能对上
7	周账	`review.md` 时间账与卡时账完成（卡时 ≤4h、花费 ≤¥10）

≥6 项通过 = 本周合格；不足 6 项，把缺的写进 Week 15 Day 1 的前 2 小时。**完成判据**：7 项里 ≥6 项打勾，`review.md` 里每一项后面都有实际数字（不许留空）。

任务 4：Week 15 预习与 gaps 交接（约 60 分钟）

1. 读 Week 15 计划的 Day 1-2 部分（DP/贪心题单 + M2 讲述版结构），把两个项目的 15 分钟讲述骨架先写在 `interview/talk-m2.md`、`talk-m3.md` 的标题行下面（只写六段小标题：动机/方案/数据/指标/失败尝试/改进）。
2. 把三档投递清单的**候选池扩到 30 家**（下周要投 ≥20 家，留 10 家做替换余量），每家的岗位链接与内推渠道先填上。
3. 在 `gaps.md` 里给每条本周暴露的洞标注「交给哪周」：八股漏洞 → Week 15 Day 3 前；未 AC 题 → Week 15 复习位；项目讲述顺序问题 → Week 15 Day 2/Day 4/Day 5；卡时补跑没做成 → Week 15 Day 6 前。

完成判据：两份讲述骨架六段标题齐；投递候选池 ≥30 家；gaps 每条都有目标周（**不许有"以后再说"的空目标**）。

任务 5：缓冲（约 70 分钟）

前面顺就提前收工。不顺就用这一小时补 fail-log 里最弱的那几条，或把 Day 5 没达标题数的题做完。**完成判据**：这一小时结束时 `gaps.md` 顶部条目数比早上少 ≥1（没减少就写清为什么）。

今日验收：`review.md` 验收表 ≥6 项打勾 + 复测三题达标 + gaps 全部有目标周。

三、常见问题与纪律

- **某天崩了怎么办？** 不熬夜追。未完成项按「天-任务号」写进 `gaps.md`，Day 7 的前 1 小时和缓冲就是给它们的。连续两天崩，砍 Day 4 任务 5（补跑脚本准备）和 Day 5 任务 4（台账跟进），**保刷题、手撕、fail-log、模拟面**。
- **单题/单点卡多久该放手？** 刷题按今天限时（25/35 分钟），到点看题解并标"抄会"，别在一道题上烧 90 分钟——那是"手感恢复期"最贵的错。非刷题的技术卡点 40 分钟无解就写 NOTES + gaps 并跳过。

- **卡时与钱**：本周上限 4h、预估 ¥6-10。**开卡前先看 `rerun.sh` 有没有写完**；跑完即关机，确认控制台显示"已关机"；**任何时候不点销毁**。超了就说明忘关机，去看计费明细，并在 `review.md` 记一笔。
- **别把模拟面当成测试**：它的唯一作用是暴露"讲不出"。第 2 次比第 1 次（Week 13）要盯的指标是**空转时长和被打断次数**，不是通过率。
- **数字纪律**：简历、fail-log、讲述稿里出现的每个指标都要有出处文件。没记录就写"量级/约"或"未量化"，**绝不写精确的假数字**——面试官追问三个细节就露馅，一露馅全盘皆输。
- **AI 工具的边界**（求职周仍然适用）：允许它解释报错、给题解思路前的提示方向、review 你写完的代码；**不允许**让 AI 直接 AC 你的题、替你写 fail-log 的真实经历、替你答八股。你面的是训练岗，现场手撕时你身边没有它。
- **诚信与合规**：博客和 repo README 继续标清哪些是复现、哪些是自己写的；引用数据集要留许可与来源。这条不会救你于面试，但会救你于 offer 之后的背调。